

3: Atividade Prática 5 — Criação e Execução de Testes Unitários e Automatizados (SonarQube)

Título: Sumário do Resultado: Criação e execução de testes unitários e automatizados

Autor: Cleydson Soares de Freitas

Curso: Especialização em DevSecOps

Instituição: FATEC Araraquara

Data: 29 de Maio de 2026

1. Introdução e Objetivos

Este relatório apresenta as evidências operacionais e a justificativa técnica da implementação de rotinas de qualidade de software através de testes unitários e análises estáticas de código (SAST) integradas ao ciclo de vida de um microserviço Go. O objetivo central do laboratório é assegurar a governança de código e a identificação precoce de vulnerabilidades ou defeitos lógicos antes da fase de implantação em produção, empregando o ecossistema integrado do GitLab CI/CD e do servidor corporativo SonarQube.

2. Execução e Configuração da Análise Estática

O servidor do SonarQube foi provisionado na infraestrutura local, permitindo o isolamento de projetos por chaves exclusivas de identificação.

1. **Criação do Projeto e Geração de Credenciais:** Efetuou-se o login com privilégios administrativos no painel central do SonarQube (<http://192.168.98.10:9000>), onde estruturou-se a criação do escopo do projeto my-golang-app e gerou-se um token de acesso secreto estático para autenticação e comunicação das esteiras de verificação.
2. **Declaração de Propriedades Locais:** Criou-se na raiz do projeto o arquivo de diretivas sonar-project.properties para delimitar caminhos de inclusão de arquivos (sonar.sources) e apontamento dos relatórios de cobertura de código (coverage.out) gerados pelos testes do Go.

Properties

sonar.projectKey=my-golang-app

sonar.projectName=my-golang-app

sonar.projectVersion=1.0

sonar.sources=.

sonar.exclusions=**/*_test.go

sonar.host.url=http://192.168.98.10:9000

sonar.token=sqp_37cf1d7b0d1f3bba567444ae0fd06ee35afdfa

3. Desenvolvimento de Testes Unitários e Integração CI/CD

Para certificar os fluxos lógicos das rotas da aplicação, implementou-se a cobertura automatizada nativa da linguagem Go.

1. **Implementação de Casos de Teste:** O arquivo `main_test.go` foi construído com base na biblioteca nativa `testing`, avaliando de forma isolada as respostas de requisições HTTP, validação de cabeçalhos de status (`http.StatusOK`) e o correto isolamento de escopo no consumo de variáveis de ambiente através da injeção de payloads controlados.
2. **Esteira Automatizada de Qualidade:** O arquivo `.gitlab-ci.yml` foi reestruturado para processar a execução de `go test` em tempo de execução de pipeline, gerando o relatório técnico que alimenta o estágio subsequente de análise estática.
 - **Evidência Aplicada (Print 1 - Terminal):** Use a imagem **image_75cb25.png**, que comprova a sincronização perfeita de todos os arquivos de testes e propriedades aceitos pelo GitLab central via Git Push.

4. Análise de Métricas e Segurança (SonarQube)

Após o processamento dos dados submetidos pelo pipeline, o painel central do SonarQube consolidou a análise e emitiu o veredito de qualidade do repositório.

1. **Cobertura de Código (Code Coverage):** Obteve-se o índice consolidado de **85.7%** de cobertura das linhas lógicas testadas, atestando a eficácia e confiabilidade dos cenários de testes unitários desenvolvidos.
2. **Segurança de Código (Security Hotspots):** A varredura estática de vulnerabilidades acusou **0%** de falhas críticas de segurança ou bugs remanescentes injetados nas dependências do microsserviço.
 - **Evidência Aplicada (Print 2 - SonarQube):** Captura de tela cheia do navegador web no painel do SonarQube exibindo o status geral **Passed** e os indicadores circulares com 85.7% de cobertura de testes.

5. Conclusão

A realização deste laboratório prático evidenciou o impacto crítico dos testes automatizados e das varreduras de código no ecossistema DevSecOps. A integração contínua do GitLab com o SonarQube provou que o código atende tanto aos requisitos de negócio lógicos quanto às diretrizes de conformidade estática contra vulnerabilidades conhecidas, blindando a esteira contra o vazamento de brechas de segurança para os servidores produtivos.

```

cleydson@Cleydson: ~/my-golang-app

Sessão  Ações  Editar  Exibir  Ajuda

cleydson@Cleydson: ~ ■ cleydson@Cleydson: ~ ■ cleydson@Cleydson: ~/my-golang-app ■ cleydson@Cleydson: ~/my-golang-app ■ cleydson@Cleydson: ~/my-golang-app ■

Auto-merging README
CONFLICT (content): Merge conflict in README
Automatic merge failed; fix conflicts and then commit the result.

(cleydson@Cleydson)~/my-golang-app
$ nano README
(cleydson@Cleydson)~/my-golang-app
$ git add README
git commit -m "Resolução do conflito de merge"
[master 60d3661] Resolução do conflito de merge

(cleydson@Cleydson)~/my-golang-app
$ # 1. Criar e alternar para a branch feature2
git checkout -b feature2

# 2. Adicionar alteração na feature2
echo "Esta é uma nova feature com o nome feature2" >> README

# 3. Commitar a alteração na branch feature2
git add README
git commit -m "Adiciona nova feature com o nome feature2"

# 4. Voltar para a branch master
git checkout master

# 5. Criar alteração concorrente na master
echo "Esta é uma segunda alteração na master" >> README

# 6. Commitar a alteração concorrente na master
git add README
git commit -m "Adiciona segunda alteração na master"

# 7. Mudar de volta para a feature2 e invocar o rebase contra a master
git checkout feature2
git rebase master
Switched to a new branch 'feature2'
[feature2 b0c0faa] Adiciona nova feature com o nome feature2
1 file changed, 1 insertion(+)
Switched to branch 'master'
Your branch is ahead of 'origin/master' by 6 commits.
(Use "git push" to publish your local commits)
[master 329ca75] Adiciona segunda alteração na master
1 file changed, 1 insertion(+)
Switched to branch 'feature2'
Auto-merging README
CONFLICT (content): Merge conflict in README
error: could not apply b0c0faa... Adiciona nova feature com o nome feature2
hint: Resolve all conflicts manually, mark them as resolved with
hint: 'git add/rm <conflicted_files>', then run 'git rebase --continue'.
hint: You can instead skip this commit: run 'git rebase --skip'.
hint: To abort and get back to the state before 'git rebase', run 'git rebase --abort'.
hint: Disable this message with 'git config set advice.mergeConflict false'
Could not apply b0c0faa... # Adiciona nova feature com o nome feature2

(cleydson@Cleydson)~/my-golang-app
$
```

```

cleydson@Cleydson: ~/my-golang-app

Sessão  Ações  Editar  Exibir  Ajuda

cleydson@Cleydson: ~ ■ cleydson@Cleydson: ~ ■ cleydson@Cleydson: ~/my-golang-app ■ cleydson@Cleydson: ~/my-golang-app ■ cleydson@Cleydson: ~/my-golang-app ■

(cleydson@Cleydson)~/my-golang-app
$

(cleydson@Cleydson)~/my-golang-app
$ # 6. Indexar e commitar a alteração concorrente na master

(cleydson@Cleydson)~/my-golang-app
$ git add README

(cleydson@Cleydson)~/my-golang-app
$ git commit -m "Adiciona alteração na master"
[master 042cfab] Adiciona alteração na master
1 file changed, 1 insertion(+)

(cleydson@Cleydson)~/my-golang-app
$

(cleydson@Cleydson)~/my-golang-app
$ # 7. Tentar realizar a fusão da branch feature para dentro da master

(cleydson@Cleydson)~/my-golang-app
$ # 1. Criar e alternar para uma nova branch paralela chamada "feature"
git checkout -b feature

# 2. Inserir uma alteração de texto simulando o desenvolvimento da nova feature
echo "Esta é uma nova feature" >> README

# 3. Indexar e commitar a alteração na branch isolada
git add README
git commit -m "Adiciona nova feature"

# 4. Retornar o ponteiro para a branch de produção principal "master"
git checkout master

# 5. Modificar a mesma linha do arquivo README de forma conflitante na master
echo "Esta é uma alteração na master" >> README

# 6. Indexar e commitar a alteração concorrente na master
git add README
git commit -m "Adiciona alteração na master"

# 7. Tentar realizar a fusão da branch feature para dentro da master
git merge feature
fatal: a branch named 'feature' already exists
[master a22089e] Adiciona nova feature
1 file changed, 1 insertion(+)
Already on 'master'
Your branch is ahead of 'origin/master' by 3 commits.
(Use "git push" to publish your local commits)
[master d1c8f22] Adiciona alteração na master
1 file changed, 1 insertion(+)
Auto-merging README
CONFLICT (content): Merge conflict in README
Automatic merge failed; fix conflicts and then commit the result.

(cleydson@Cleydson)~/my-golang-app
$ []
```

```

Sessão  Ações  Editar  Exibir  Ajuda
cleydson@Cleydson: ~ -  cleydson@Cleydson: ~ -  cleydson@Cleydson: ~/my-golang-app  cleydson@Cleydson: ~/my-golang-app  cleydson@Cleydson: ~/my-golang-app
[cleydson@Cleydson] ~/my-golang-app
$ git tag v1.0
git tag
v1.0
git add README
git commit -m "Adiciona nova alteração"
v1.0
[master 9a7655d] Adiciona nova alteração
1 file changed, 1 insertion(+)
[cleydson@Cleydson] ~/my-golang-app
$ git tag v1.1
git tag
v1.1
git checkout v1.0
git checkout v1.1
v1.0
v1.1
Note: switching to 'v1.0'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at df13b08 InicioFeature0!
Arquivo Modificado
Previous HEAD position was df13b08 InicioFeature0!
HEAD is now at 9a7655d Adiciona nova alteração
[cleydson@Cleydson] ~/my-golang-app
$ 
```

```

Sessão  Ações  Editar  Exibir  Ajuda
cleydson@Cleydson: ~ -  cleydson@Cleydson: ~ -  cleydson@Cleydson: ~/my-golang-app  cleydson@Cleydson: ~/my-golang-app  cleydson@Cleydson: ~/my-golang-app
$ # 1. Alocar as alterações pendentes da feature3 no stash para limpar a árvore de trabalho
git stash
# 2. Mudar para o branch master livremente agora que o diretório está limpo
git checkout master
# 3. Gerar a colisão concorrente adicionando a terceira linha na master
echo "Esta é uma terceira alteração na master" >> README
# 4. Indexar e efetuar o commit dessa alteração diretamente na master
git add README
git commit -m "Adiciona terceira alteração na master"
# 5. Retornar o ponteiro para o branch feature3
git checkout feature3
# 6. Descarregar o estado suspenso da pilha para estourar o conflito definitivo
git stash pop
Saved working directory and index state WIP on feature3: 7458c7b Adiciona nova feature com o nome feature2
Switched to branch 'master'
Your branch is ahead of 'origin/master' by 7 commits.
(use 'git push' to publish your local commits)
[master 7ae27fc] Adiciona terceira alteração na master
1 file changed, 1 insertion(+)
Switched to branch 'feature3'
On branch feature3
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README
no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (b7ddf6a9db8e8602ab58cceb21ceea7c3b2b516)
[cleydson@Cleydson] ~/my-golang-app
$ git add README
git commit -m "Resolve conflito com git stash"
git push origin feature3
[feature3 e4ea397] Resolve conflito com git stash
1 file changed, 1 insertion(+)
Username for 'http://localhost': root
Password for 'http://localhost':
Enumerating objects: 29, done.
Counting objects: 100% (29/29), done.
Delta compression using up to 16 threads
Compressing objects: 100% (26/26), done.
Writing objects: 100% (27/27), 2.86 KiB | 975.00 KiB/s, done.
Total 27 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: To create a merge request for feature3, visit:
remote:   http://localhost:root/my-golang-app/-/merge_requests/new?merge_request%5Bsource_branch%5D=feature3
remote:
To http://localhost:root/my-golang-app.git
 * [new branch]   feature3 -> feature3
[cleydson@Cleydson] ~/my-golang-app
$ 
```